

Story 15: 查询患者血压的变化趋势

1. 1. Story功能描述

1.1. 系统名称

生命体征监测系统 - 血压趋势监测模块

1.2. 类型

医疗数据分析功能

1.3. 在项目中的地位

作为生命体征监测系统的核心分析功能之一，负责患者血压数据的趋势分析，为医护人员提供决策支持。

1.4. 任务目标

- 展示患者血压数据的时间序列变化趋势
- 计算并显示收缩压和舒张压的统计数据
- 支持多时间范围的数据查询
- 提供数据可视化分析

2. 2. 详细设计

2.1. 2.1 界面

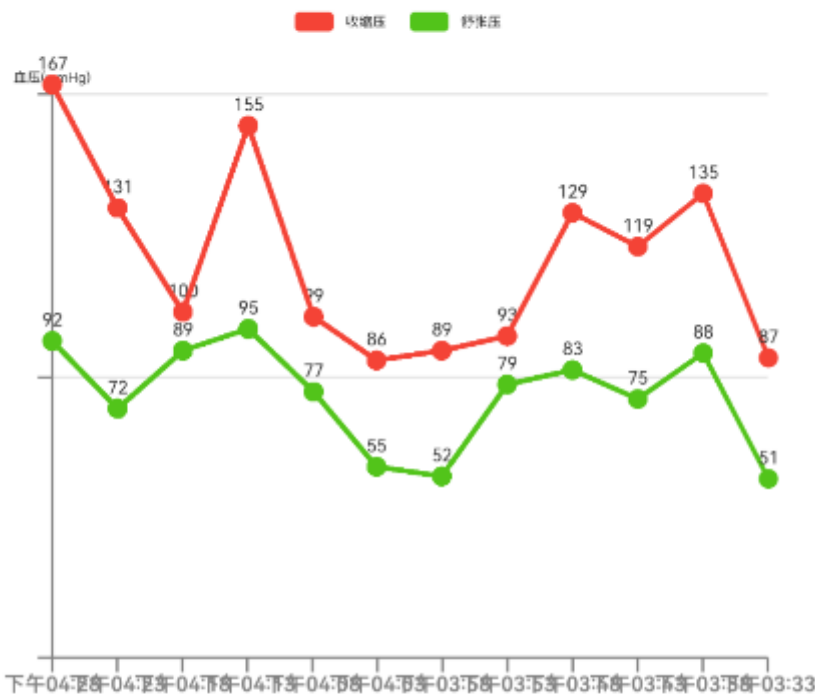
11:00

100

< 血压趋势

血压趋势

2025/4/18



167 mmHg

最高值

116 mmHg

平均值

86 mmHg

最低值

95 mmHg

最高值

76 mmHg

平均值

51 mmHg

最低值

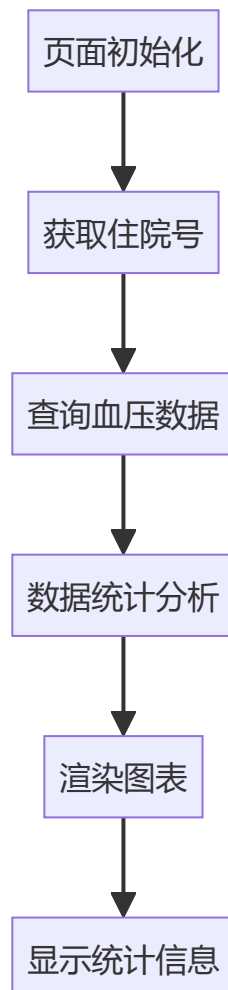
查看详细数据

```

1 Column() {
2   // 顶部导航栏
3   Row() {
4     Image($r('app.media.ic_public_arrow_left'))
5     Text('血压趋势')
6   }
7
8   // 图表区域
9   McLineChart({
10    xAxis: { type: 'category', data: timeData },
11    series: [
12      { name: '收缩压', data: systolicData },
13      { name: '舒张压', data: diastolicData }
14    ]
15  })
16
17  // 统计数据区
18  Row() {
19    StatisticCard('最高值', `${maxSystolic}/${maxDiastolic}`)
20    StatisticCard('平均值', `${avgSystolic}/${avgDiastolic}`)
21    StatisticCard('最低值', `${minSystolic}/${minDiastolic}`)
22  }
23 }

```

2.2. 2.2 内部处理流程



2.3. 2.3 数据流

输入:

```
1 interface BloodPressureData {
2   systolic: number;    // 收缩压
3   diastolic: number;   // 舒张压
4   measureTime: string; // 测量时间
5   hospitalId: string;  // 住院号
6 }
```

输出:

```
1 interface BloodPressureStats {
2   highest: string;    // "收缩压/舒张压"
3   lowest: string;     // "收缩压/舒张压"
4   average: string;    // "收缩压/舒张压"
5 }
```

2.4. 2.4 算法描述

```
1 function calculateBloodPressureStats(data: BloodPressureData[]):
  BloodPressureStats {
2   // 初始化统计变量
3   let maxSystolic = data[0].systolic;
4   let minSystolic = data[0].systolic;
5   let maxDiastolic = data[0].diastolic;
6   let minDiastolic = data[0].diastolic;
7   let sumSystolic = 0;
8   let sumDiastolic = 0;
9
10  // 遍历计算
11  for (const record of data) {
12    maxSystolic = Math.max(maxSystolic, record.systolic);
13    minSystolic = Math.min(minSystolic, record.systolic);
14    maxDiastolic = Math.max(maxDiastolic, record.diastolic);
15    minDiastolic = Math.min(minDiastolic, record.diastolic);
16    sumSystolic += record.systolic;
17    sumDiastolic += record.diastolic;
18  }
19
20  // 计算平均值
21  const avgSystolic = Math.round(sumSystolic / data.length);
22  const avgDiastolic = Math.round(sumDiastolic / data.length);
23
24  return {
25    highest: `${maxSystolic}/${maxDiastolic}`,
26    lowest: `${minSystolic}/${minDiastolic}`,
27    average: `${avgSystolic}/${avgDiastolic}`
28  };
29 }
```

2.5. 2.5 外部接口

```
1 interface VitalSignsDB {
2     // 查询指定住院号患者的血压数据
3     getBloodPressuresByHospitalId(hospitalId: string):
4     Promise<BloodPressureData[]>;
5 }
6 interface RouterInterface {
7     // 页面导航接口
8     pushUrl(params: {
9         url: string;
10        params: {
11            hospitalId: string;
12            type: 'bloodPressure';
13        };
14    }): void;
15 }
```

2.6. 2.6 其他设计考虑

2.6.1. 性能设计

- 使用 McLineChart 组件优化大数据量渲染
- 实现数据分页加载机制
- 缓存已加载的数据减少查询

2.6.2. 安全设计

- 数据访问权限控制
- 敏感信息加密存储
- 操作日志记录

2.6.3. 可靠性设计

- 网络异常重试机制
- 数据加载失败提示
- 异常数据处理